

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Vikas Gautam	Bangalore (India)	vikasgtm@gmail.com	
José Martín Muñoz	Madrid (Spain)	13josemart@gmail.com	
Saurav	India	sauraviiser@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1	Vikas Gautam
Team member 2	Jose Martin
Team member 3	Saurav

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

N/A

Group Work Project 2

Step 1

1.a. Forward algorithm and backward algorithm

Forward Algorithm Pseudocode

Compute $P(O|\lambda)$, the probability of an observed sequence given an HMM $\lambda=(\Pi,A,B)$.

Input:

$O = [o_1, o_2, \dots, o_T]$ # Observed sequence
 $S = [s_1, s_2, \dots, s_N]$ # Hidden states
 $A = [a_{ij}]$ # Transition matrix: $a_{ij} = P(q_{t+1}=s_j | q_t=s_i)$
 $B = [b_j(o_t)]$ # Emission matrix: $b_j(o_t) = P(o_t | q_t = s_j)$
 $\pi = [\pi_i]$ # Initial state distribution

Output:

Alpha matrix ($T \times N$): $\alpha_t(i) = P(\text{partial sequence up to } t, \text{ ending in state } i)$

Algorithm:

1. Initialization:

for each state i :

$$\alpha_1(i) \leftarrow \pi_i * b_i(o_1)$$

2. Recursion:

for $t = 2$ to T :

for each state i :

$$\alpha_t(i) \leftarrow b_i(o_t) * \sum_j [\alpha_{t-1}(j) * a_{ji}]$$

3. Termination:

$$P(O | \lambda) = \sum_i [\alpha_T(i)]$$

Toy example:

States: Rainy, Sunny

Observations: walk, shop, clean

Group Number: 10177

Observed sequence (O): [walk, shop, clean]

Initial probabilities (π): $\pi = [0.6 \text{ (Rainy)}, 0.4 \text{ (Sunny)}]$

Transition matrix (A):

A =

$$\begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix}$$

(Rows represent "from" states, columns represent "to" states)

Emission probabilities (B):

Rainy: $P(\text{walk}) = 0.1$, $P(\text{shop}) = 0.4$, $P(\text{clean}) = 0.5$ Sunny: $P(\text{walk}) = 0.6$, $P(\text{shop}) = 0.3$, $P(\text{clean}) = 0.1$

Forward Algorithm Computation

Step 1: Initialization ($t = 1$)

$$\alpha_1(\text{Rainy}) = \pi(\text{Rainy}) \times b_{\text{Rainy}}(\text{walk}) = 0.6 \times 0.1 = 0.06$$

$$\alpha_1(\text{Sunny}) = \pi(\text{Sunny}) \times b_{\text{Sunny}}(\text{walk}) = 0.4 \times 0.6 = 0.24$$

Step 2: Recursion ($t = 2$)

$$\begin{aligned} \alpha_2(\text{Rainy}) &= b_{\text{Rainy}}(\text{shop}) \times [\alpha_1(\text{Rainy}) \times a_{\text{Rainy} \rightarrow \text{Rainy}} + \alpha_1(\text{Sunny}) \times a_{\text{Sunny} \rightarrow \text{Rainy}}] \\ &= 0.4 \times [0.06 \times 0.7 + 0.24 \times 0.4] \\ &= 0.4 \times [0.042 + 0.096] \\ &= 0.4 \times 0.138 = 0.0552 \end{aligned}$$

$$\begin{aligned} \alpha_2(\text{Sunny}) &= b_{\text{Sunny}}(\text{shop}) \times [\alpha_1(\text{Rainy}) \times a_{\text{Rainy} \rightarrow \text{Sunny}} + \alpha_1(\text{Sunny}) \times a_{\text{Sunny} \rightarrow \text{Sunny}}] \\ &= 0.3 \times [0.06 \times 0.3 + 0.24 \times 0.6] \\ &= 0.3 \times [0.018 + 0.144] \\ &= 0.3 \times 0.162 = 0.0486 \end{aligned}$$

Step 3: Recursion ($t = 3$)

$$\begin{aligned} \alpha_3(\text{Rainy}) &= b_{\text{Rainy}}(\text{clean}) \times [\alpha_2(\text{Rainy}) \times a_{\text{Rainy} \rightarrow \text{Rainy}} + \alpha_2(\text{Sunny}) \times a_{\text{Sunny} \rightarrow \text{Rainy}}] \\ &= 0.5 \times [0.0552 \times 0.7 + 0.0486 \times 0.4] \\ &= 0.5 \times [0.03864 + 0.01944] \\ &= 0.5 \times 0.05808 = 0.02904 \end{aligned}$$

$$\begin{aligned} \alpha_3(\text{Sunny}) &= b_{\text{Sunny}}(\text{clean}) \times [\alpha_2(\text{Rainy}) \times a_{\text{Rainy} \rightarrow \text{Sunny}} + \alpha_2(\text{Sunny}) \times a_{\text{Sunny} \rightarrow \text{Sunny}}] \\ &= 0.1 \times [0.0552 \times 0.3 + 0.0486 \times 0.6] \\ &= 0.1 \times [0.01656 + 0.02916] \\ &= 0.1 \times 0.04572 = 0.004572 \end{aligned}$$

Final result:

$$P(O \mid \lambda) = \alpha_3(\text{Rainy}) + \alpha_3(\text{Sunny}) = 0.02904 + 0.004572 = 0.033612$$

Backward algorithm:

Pseudocode

Input:

Same as forward algorithm

Output:

Beta matrix (T x N): $\beta_t(i)$

Algorithm:

1. Initialization:

for each state i:

$$\beta_T(i) \leftarrow 1$$

2. Recursion:

for t = T-1 down to 1:

for each state i:

$$\beta_t(i) \leftarrow \text{sum over } j [a_{ij} * b_j(o_{t+1}) * \beta_{t+1}(j)]$$

3. Termination:

$$P(O \mid \lambda) = \text{sum over } i [\pi_i * b_i(o_1) * \beta_1(i)]$$

Toy example:

States: Rainy, Sunny

Observations: walk, shop, clean

Observed sequence (O): [walk, shop, clean]

Initial probabilities (π): $\pi = [0.6 \text{ (Rainy)}, 0.4 \text{ (Sunny)}]$

Transition matrix (A):

A =

[
0.7 0.3
0.4 0.6
]

(Rows represent "from" states, columns represent "to" states)

Group Number: 10177

Emission probabilities (B):

Rainy: $P(\text{walk}) = 0.1$, $P(\text{shop}) = 0.4$, $P(\text{clean}) = 0.5$ Sunny: $P(\text{walk}) = 0.6$, $P(\text{shop}) = 0.3$, $P(\text{clean}) = 0.1$

Backward Algorithm Computation

Step 1: Initialization ($t = 3$) $\beta_3(\text{Rainy}) = \beta_3(\text{Sunny}) = 1$ (we assume the probability of any final observation = 1)Step 2: Recursion ($t = 2$)

$$\begin{aligned}\beta_2(\text{Rainy}) &= a_{\text{Rainy} \rightarrow \text{Rainy}} \times b_{\text{Rainy}}(\text{clean}) \times \beta_3(\text{Rainy}) + a_{\text{Rainy} \rightarrow \text{Sunny}} \times \\ &b_{\text{Sunny}}(\text{clean}) \times \beta_3(\text{Sunny}) \\ &= 0.7 \times 0.5 \times 1 + 0.3 \times 0.1 \times 1 = 0.35 + 0.03 = 0.38\end{aligned}$$

$$\begin{aligned}\beta_2(\text{Sunny}) &= a_{\text{Sunny} \rightarrow \text{Rainy}} \times b_{\text{Rainy}}(\text{clean}) \times \beta_3(\text{Rainy}) + a_{\text{Sunny} \rightarrow \text{Sunny}} \times \\ &b_{\text{Sunny}}(\text{clean}) \times \beta_3(\text{Sunny}) \\ &= 0.4 \times 0.5 \times 1 + 0.6 \times 0.1 \times 1 = 0.2 + 0.06 = 0.26\end{aligned}$$

Step 3: Recursion ($t = 1$)

$$\begin{aligned}\beta_1(\text{Rainy}) &= a_{\text{Rainy} \rightarrow \text{Rainy}} \times b_{\text{Rainy}}(\text{shop}) \times \beta_2(\text{Rainy}) + a_{\text{Rainy} \rightarrow \text{Sunny}} \times b_{\text{Sunny}}(\text{shop}) \\ &\times \beta_2(\text{Sunny}) \\ &= 0.7 \times 0.4 \times 0.38 + 0.3 \times 0.3 \times 0.26 \\ &= 0.1064 + 0.0234 = 0.1298\end{aligned}$$

$$\begin{aligned}\beta_1(\text{Sunny}) &= a_{\text{Sunny} \rightarrow \text{Rainy}} \times b_{\text{Rainy}}(\text{shop}) \times \beta_2(\text{Rainy}) + a_{\text{Sunny} \rightarrow \text{Sunny}} \times \\ &b_{\text{Sunny}}(\text{shop}) \times \beta_2(\text{Sunny}) \\ &= 0.4 \times 0.4 \times 0.38 + 0.6 \times 0.3 \times 0.26 \\ &= 0.0608 + 0.0468 = 0.1076\end{aligned}$$

Final result:

$$\begin{aligned}P(O \mid \lambda) &= \pi(\text{Rainy}) \times b_{\text{Rainy}}(\text{walk}) \times \beta_1(\text{Rainy}) + \pi(\text{Sunny}) \times b_{\text{Sunny}}(\text{walk}) \times \beta_1(\text{Sunny}) \\ &= 0.6 \times 0.1 \times 0.1298 + 0.4 \times 0.6 \times 0.1076 \\ &= 0.007788 + 0.025824 = 0.033612\end{aligned}$$

1.b. Backward Viterbi algorithm

The **Viterbi algorithm** finds the **most likely sequence of hidden states** (also called the **Viterbi path**) that could generate a given sequence of observations from a Hidden Markov Model (HMM).

While the standard Viterbi is a forward recursion that stores backpointers, WE can also structure the logic as a **backward traceback** after computing the maximum probabilities. This is what's referred to as the "backward Viterbi".

Pseudocode:

Input:

$O = [o_1, o_2, \dots, o_T]$ # Observation sequence
 $S = [s_1, s_2, \dots, s_N]$ # Hidden states
 $A = [a_{ij}]$ # Transition matrix
 $B = [b_j(o_t)]$ # Emission matrix
 $\pi = [\pi_i]$ # Initial state distribution

Output:

Most likely hidden state sequence $q^* = [q_1^*, q_2^*, \dots, q_T^*]$

Algorithm:

1. Initialization ($t = 1$):

For each state i :

$V[1][i] \leftarrow \pi[i] \times B[i][o_1]$
 $\text{Backpointer}[1][i] \leftarrow \text{null}$

2. Recursion ($t = 2$ to T):

For each state i :

$V[t][i] \leftarrow \max_j [V[t-1][j] \times A[j][i]] \times B[i][o_t]$
 $\text{Backpointer}[t][i] \leftarrow \arg\max_j [V[t-1][j] \times A[j][i]]$

3. Termination:

$q_T^* \leftarrow \arg\max_i V[T][i]$
Backtrack from $t = T$ to 1 using Backpointer to recover $q_1^*, \dots, q_T^*$

Toy example:

Same data as before and:

Step 1: Initialization ($t = 1$)

$V_1(\text{Rainy}) = 0.6 \times 0.1 = 0.06$

$V_1(\text{Sunny}) = 0.4 \times 0.6 = 0.24$

$\text{Backpointer}_1 = [\text{null}, \text{null}]$

Step 2: Recursion ($t = 2$)

$$\begin{aligned} V_2(\text{Rainy}) &= \max(\\ &\quad 0.06 \times 0.7 = 0.042, \\ &\quad 0.24 \times 0.4 = 0.096 \\ &) \times 0.4 = 0.096 \times 0.4 = 0.0384 \\ \text{Backpointer}_2(\text{Rainy}) &= \text{Sunny} \end{aligned}$$

$$\begin{aligned} V_2(\text{Sunny}) &= \max(\\ &\quad 0.06 \times 0.3 = 0.018, \\ &\quad 0.24 \times 0.6 = 0.144 \\ &) \times 0.3 = 0.144 \times 0.3 = 0.0432 \\ \text{Backpointer}_2(\text{Sunny}) &= \text{Sunny} \end{aligned}$$

Step 3: Recursion ($t = 3$)

$$\begin{aligned} V_3(\text{Rainy}) &= \max(\\ &\quad 0.0384 \times 0.7 = 0.02688, \\ &\quad 0.0432 \times 0.4 = 0.01728 \\ &) \times 0.5 = 0.02688 \times 0.5 = 0.01344 \\ \text{Backpointer}_3(\text{Rainy}) &= \text{Rainy} \end{aligned}$$

$$\begin{aligned} V_3(\text{Sunny}) &= \max(\\ &\quad 0.0384 \times 0.3 = 0.01152, \\ &\quad 0.0432 \times 0.6 = 0.02592 \\ &) \times 0.1 = 0.02592 \times 0.1 = 0.002592 \\ \text{Backpointer}_3(\text{Sunny}) &= \text{Sunny} \end{aligned}$$

Step 4: Termination and Path Backtracking

Final state = $\text{argmax}(V_3) = \text{Rainy}$ ($0.01344 > 0.002592$)

Backtrack:

$$\begin{aligned} t = 3: & \text{Rainy} \\ t = 2: & \text{Backpointer}_3(\text{Rainy}) = \text{Rainy} \\ t = 1: & \text{Backpointer}_2(\text{Rainy}) = \text{Sunny} \end{aligned}$$

Most Likely Hidden State Path (q^*):

Sunny $\rightarrow$ Rainy $\rightarrow$ Rainy

1.c. Baum-Welch algorithm

The **Baum-Welch algorithm** is an instance of the **Expectation-Maximization (EM)** method used to **learn the optimal parameters** of a Hidden Markov Model $\lambda=(\pi,A,B)$ from observed sequences when the hidden states are unknown.

- The **Baum-Welch algorithm** learns optimal HMM parameters from data.
- It does **not require knowing the true hidden state paths**.
- It uses **expectations** (γ, ξ) to update π, A, B iteratively

Baum-Welch Algorithm: Pseudocode

Input:

- $O = [o_1, o_2, \dots, o_T]$ # Observation sequence
- N = number of states
- M = number of observation symbols
- Initial estimates: A (transitions), B (emissions), π (initial distribution)
- Convergence threshold ϵ

Output:

- Updated A, B, π that maximize $P(O | \lambda)$

Algorithm:

1. Repeat until convergence:

- FORWARD step: Compute $\alpha_t(i)$ for all t and states i
- BACKWARD step: Compute $\beta_t(i)$ for all t and states i
- Compute $\gamma_t(i)$ = probability of being in state i at time t :
$$\gamma_t(i) = [\alpha_t(i) * \beta_t(i)] / P(O | \lambda)$$
- Compute $\xi_t(i, j)$ = probability of transitioning from state i to j at time t :
$$\xi_t(i, j) = [\alpha_t(i) * A[i][j] * B[j][o_{t+1}] * \beta_{t+1}(j)] / P(O | \lambda)$$
- Re-estimate π :
$$\pi[i] = \gamma_1(i)$$
- Re-estimate transition probabilities A :
$$A[i][j] = \sum_{t=1}^{T-1} \xi_t(i, j) / \sum_{t=1}^{T-1} \gamma_t(i)$$
- Re-estimate emission probabilities B :
For each symbol k :

Group Number: 10177

$$B[i][k] = \sum_{t=1}^T \gamma_t(i) \text{ where } o_t = k / \sum_{t=1}^T \gamma_t(i)$$

2. Return updated A, B, π **Toy example:**

States: Rainy, Sunny

Observations: walk, shop

Observation sequence: O = [walk, shop]

Initial estimates:

$$\pi = [0.5, 0.5]$$

$$A = \begin{bmatrix} 0.6 & 0.4 \\ 0.5 & 0.5 \end{bmatrix}$$

B =

Rainy: walk=0.7, shop=0.3

Sunny: walk=0.4, shop=0.6

Step 1: Forward Pass (α)

At t = 1:

$$\alpha_1(\text{Rainy}) = 0.5 \times 0.7 = 0.35$$

$$\alpha_1(\text{Sunny}) = 0.5 \times 0.4 = 0.2$$

At t = 2:

$$\alpha_2(\text{Rainy}) = 0.3 \times (0.35 \times 0.6 + 0.2 \times 0.5) = 0.3 \times (0.21 + 0.1) = 0.3 \times 0.31 = 0.093$$

$$\alpha_2(\text{Sunny}) = 0.6 \times (0.35 \times 0.4 + 0.2 \times 0.5) = 0.6 \times (0.14 + 0.1) = 0.6 \times 0.24 = 0.144$$

$$P(O | \lambda) = \alpha_2(\text{Rainy}) + \alpha_2(\text{Sunny}) = 0.093 + 0.144 = 0.237$$

Step 2: Backward Pass (β)

At t = 2:

$$\beta_2(\text{Rainy}) = \beta_2(\text{Sunny}) = 1$$

At t = 1:

$$\beta_1(\text{Rainy}) = 0.6 \times 0.3 + 0.4 \times 0.6 = 0.18 + 0.24 = 0.42$$

$$\beta_1(\text{Sunny}) = 0.5 \times 0.3 + 0.5 \times 0.6 = 0.15 + 0.3 = 0.45$$

Step 3: Compute γ (posterior state probability)

Group Number: 10177

$$\gamma_1(\text{Rainy}) = (0.35 \times 0.42) / 0.237 \approx 0.619$$

$$\gamma_1(\text{Sunny}) = (0.2 \times 0.45) / 0.237 \approx 0.380$$

$$\gamma_2(\text{Rainy}) = 0.093 / 0.237 \approx 0.392$$

$$\gamma_2(\text{Sunny}) = 0.144 / 0.237 \approx 0.608$$

Step 4: Re-estimate π

$$\pi' = [\gamma_1(\text{Rainy}), \gamma_1(\text{Sunny})] = [0.619, 0.380]$$

Step 5: Re-estimate A

$$\text{Denominator for Rainy} = \gamma_1(\text{Rainy}) = 0.619$$

$$\xi_1(\text{Rainy} \rightarrow \text{Rainy}) = (0.35 \times 0.6 \times 0.3 \times 1) / 0.237 = 0.063 / 0.237 \approx 0.266$$

$$\xi_1(\text{Rainy} \rightarrow \text{Sunny}) = (0.35 \times 0.4 \times 0.6 \times 1) / 0.237 = 0.084 / 0.237 \approx 0.354$$

$$A'[\text{Rainy}][\text{Rainy}] = 0.266 / 0.619 \approx 0.43$$

$$A'[\text{Rainy}][\text{Sunny}] = 0.354 / 0.619 \approx 0.57$$

(Similarly for Sunny)

Step 6: Re-estimate B

For Rainy:

$$B'[\text{Rainy}][\text{walk}] = \gamma_{t=1}(\text{Rainy}) / \text{total } \gamma(\text{Rainy}) = 0.619 / (0.619 + 0.392) \approx 0.612$$

$$B'[\text{Rainy}][\text{shop}] = \gamma_{t=2}(\text{Rainy}) / \text{total } \gamma(\text{Rainy}) = 0.392 / (0.619 + 0.392) \approx 0.388$$

(Similarly for Sunny)

Step 2

2.a. Student A will identify and illustrate examples of bull regimes.

A bull regime e.g. in stock refers to a period (range) during which asset prices rise consistently overall, often backed by strong investor confidence, positive market sentiments, news of economic growth, or good quarterly results of business or finding of crude oil reserves underground and more job opportunities. The margin is quite significant between the end of period value and start of period value.

Characteristics:

- Positive market sentiment and investor optimism
- Higher highs and higher lows in price movement
- Strong earnings growth and economic expansion
- Increasing volumes in stock purchases

- Low market volatility (in some phases)
- Expectations that the momentum will continue for some more time

1. Dot-com Bull Market (1995–2000):

- Fueled by internet and technology stocks.
- NASDAQ rose more than 400% from 1995 to its peak in March 2000.
- Characterized by high investor optimism and tech IPOs.

2. COVID-Recovery Bull Run (Mar 2020–Dec 2021):

- Market rebounded rapidly after the pandemic crash.
- Fueled by fiscal stimulus, Fed liquidity, and recovery optimism.
- S&P 500 rose over 100% from March 2020 to end of 2021.

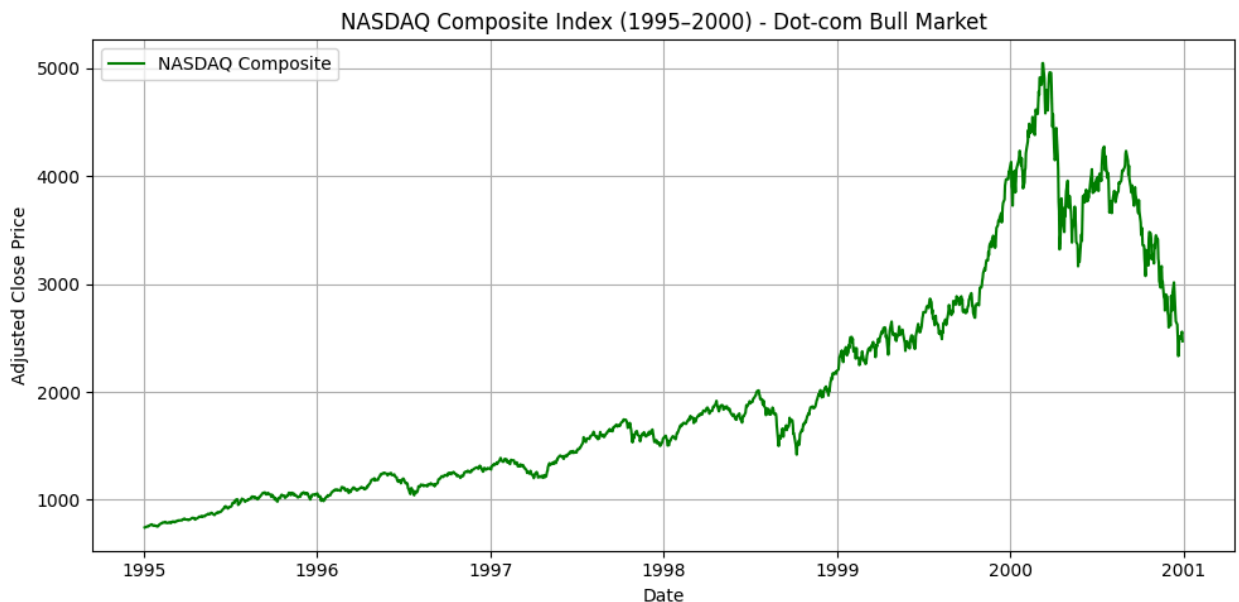


Figure 1. Bull regimes.

2.b. Student B will identify and illustrate examples of bear regimes.

A **bear regime** is a period characterized by **sustained declines in asset prices**, typically in the environment of pessimism, economic contraction, and declining investor confidence. A bear market can be defined as a **decline of 20% or more** from recent highs.

Characteristics:

- Negative market sentiment and fear-based selling

- Lower highs and lower lows in asset prices
- Rising volatility and falling earnings expectations
- Often triggered by economic shocks, recessions, or tightening monetary policy

1. Dot-com Crash (2000–2002):

- NASDAQ fell ~78% from March 2000 peak to October 2002.
- Triggered by overvalued tech stocks collapsing post-bubble.
- Main reason was companies having no profits or revenues, they were expecting some future earnings but never happened
- Many firms used their IPO proceedings in marketing rather than R&D or operations

2. Global Financial Crisis (2007–2009):

- S&P 500 dropped ~57% between October 2007 and March 2009.
- Caused by the collapse of the housing market and credit crisis.

3. COVID-19 crash (Feb-Mar 2020):

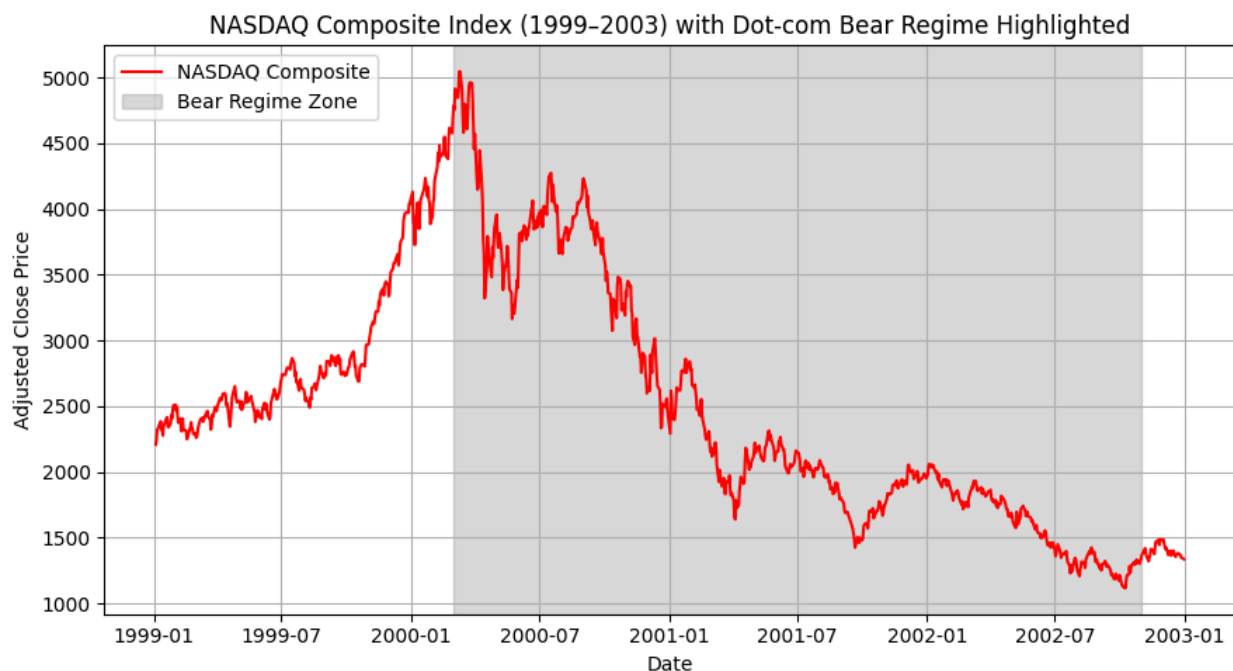


Figure 2. Bear regimes.

2.c. Student C will identify and illustrate examples of stagnant regimes

A **stagnant regime** (also called a **sideways market** or **range-bound market**) occurs when asset prices **oscillate within a narrow band** over an extended period without showing a clear upward or downward trend.

Characteristics:

- Price fluctuates between a support and resistance zone
- Lack of strong directional momentum
- Low average return, often with low or moderate volatility
- Investors may exhibit indecision or await macroeconomic clarity
- Often follows or precedes strong bull/bear trends

1. Dow Jones (1965–1982):

- The Dow fluctuated mostly between 600 and 1000 for **17 years**, adjusting for inflation.
- A lost decade with negligible real returns.

2. S&P 500 (2000–2013):

- From March 2000 to early 2013, the S&P 500 underwent two crashes and recoveries.
- Index moved between ~750 and ~1500 for 13 years — a stagnant regime in real terms.

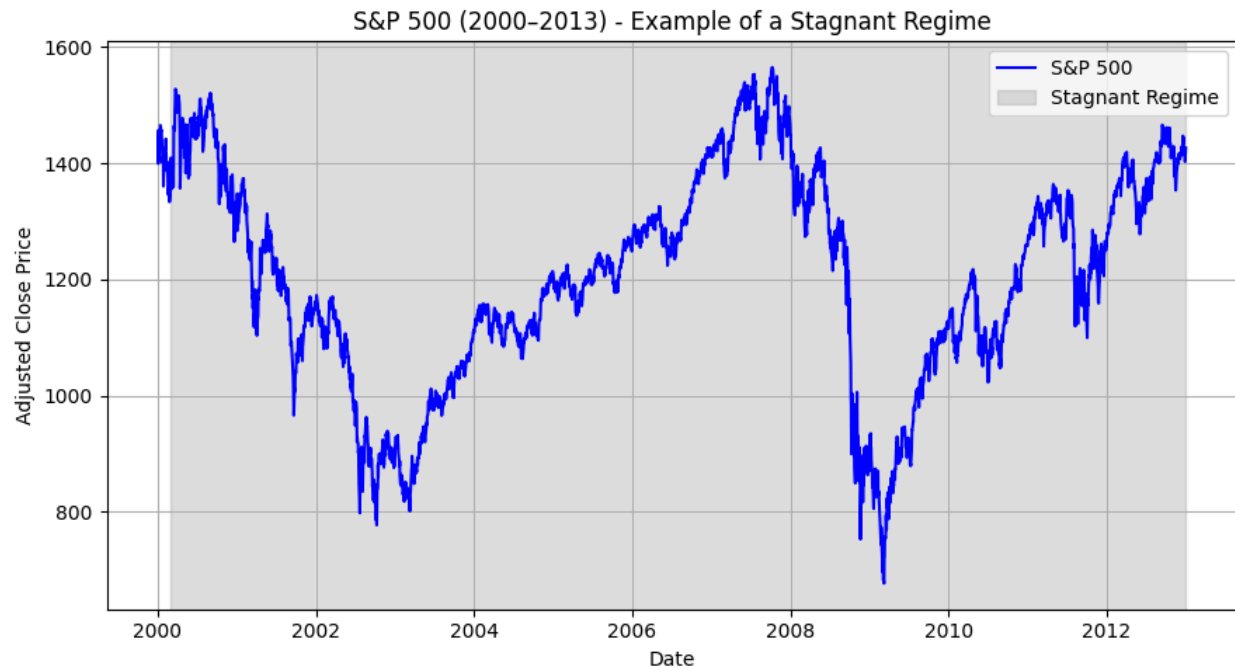


Figure 3. Stagnant regimes.

Now, regarding oil prices: using a 3-state Hidden Markov Model (HMM) trained on daily log returns of WTI crude oil prices (2010–2025), we identified three latent regimes interpreted as:

- **Bull:** Short bursts of strong positive momentum (State 1)
- **Bear:** Sustained declines with high volatility (State 2)
- **Stagnant:** Long periods of sideways or moderate upward drift (State 0)

The regime statistics are summarized below:

Regime	Days	Avg Daily Return
Bull	89	+0.0059
Bear	1346	−0.0016
Stagnant	2334	+0.00093

Figure 4 displays these results:

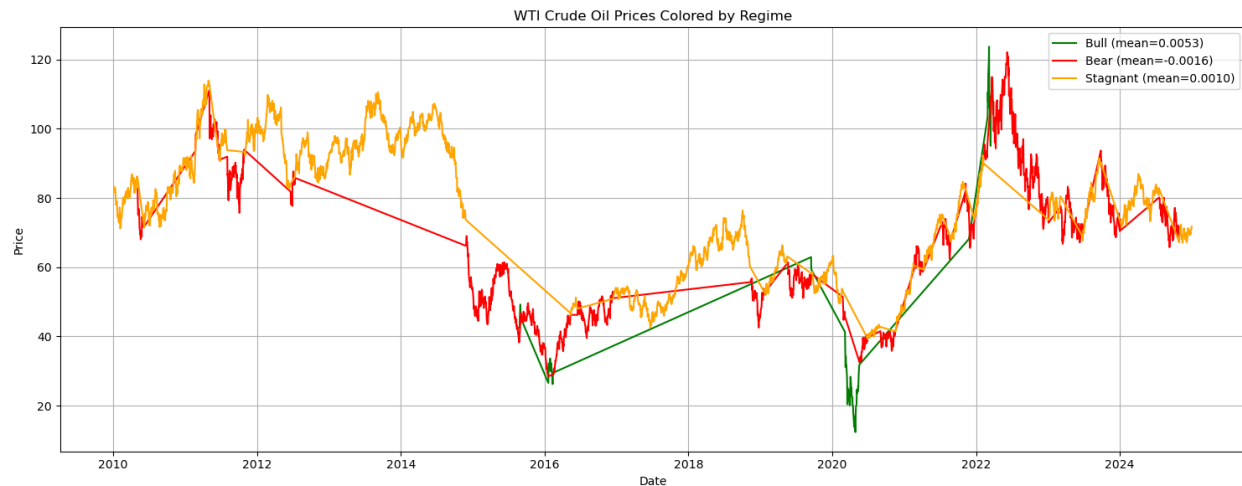


Figure 4. Oil regimes.

Although the Hidden Markov Model does a good job segmenting the oil price series into three different regimes based on return patterns, its interpretation has some limitations.

The model captures bear markets quite well. This regime, which has the most negative returns, aligns clearly with major downturns like the 2014–2016 oil collapse and the COVID-19 crash (although to some extent there are parts identified as bull regimes). These periods are strongly defined and the model picks them up effectively.

However, the bull regime is less convincing. Even though it shows the highest average returns, it only appears for a very small number of days (just 89). This suggests that the model might be too cautious in labeling bullish trends, and could be lumping short-lived rallies into other regimes instead, especially the stagnant one.

The stagnant regime dominates most of the timeline. While it has low daily returns, it seems to absorb a mix of sideways, flat, and even mildly bullish movements. That might be due to its long duration and relatively stable behavior.

In short, while the HMM is quite effective at spotting bearish conditions, it's less successful at isolating bullish ones, which often get blended into more neutral regimes. This could potentially be improved by adjusting the model (e.g., adding more states or features) to better capture different types of positive momentum.

Step 3

A Hidden Markov Model (HMM) is a probabilistic model used to describe systems that evolve over time and generate observable outputs that depend on unobservable (hidden) internal states. The key idea is that the system transitions between these hidden states according to a Markov process, and each state probabilistically emits an observable symbol or value.

Formally, an HMM consists of:

- A finite set of hidden states, $S = \{s_1, s_2, \dots, s_N\}$, which the system can occupy.
- An initial state distribution, $\pi = \{\pi_i\}$, where $\pi_i = P(q_1 = s_i)$ gives the probability of starting in state s_i .
- A state transition matrix, $A = \{a_{ij}\}$, where $a_{ij} = P(q_{t+1} = s_j | q_t = s_i)$ is the probability of transitioning from state s_i to state s_j .
- An observation (emission) model, which defines the probability of observing a symbol o_t given the system is in state s_i . This can be discrete (with a probability table) or continuous (with a distribution like Gaussian).

The key thing about HMMs is that they assume the system's next state only depends on its current one, not on the full history. Also, whatever we observe depends only on the current state. That's why it's called a *Markov* model.

People use HMMs in a lot of areas where data comes in sequences, like speech recognition, genetics, and finance. In our case, for financial time series, HMMs are helpful for figuring out hidden "regimes" in the market, like when prices are generally rising (bull markets), falling (bear markets), or kind of flat (stagnant). Even though we can't directly observe these regimes, we can guess which one we're in by looking at patterns in returns over time.

Since we don't actually know the real sequence of hidden states, we need to infer it using some standard algorithms. The Forward-Backward algorithm helps us calculate probabilities over time, Viterbi is used to find the most likely sequence of states, and Baum-Welch is used to train the model, basically adjusting the parameters so they best fit the data we have.

Step 4

Each student wrote about their respective topic. The three papers combined forming one narrative can be found in Step 5.

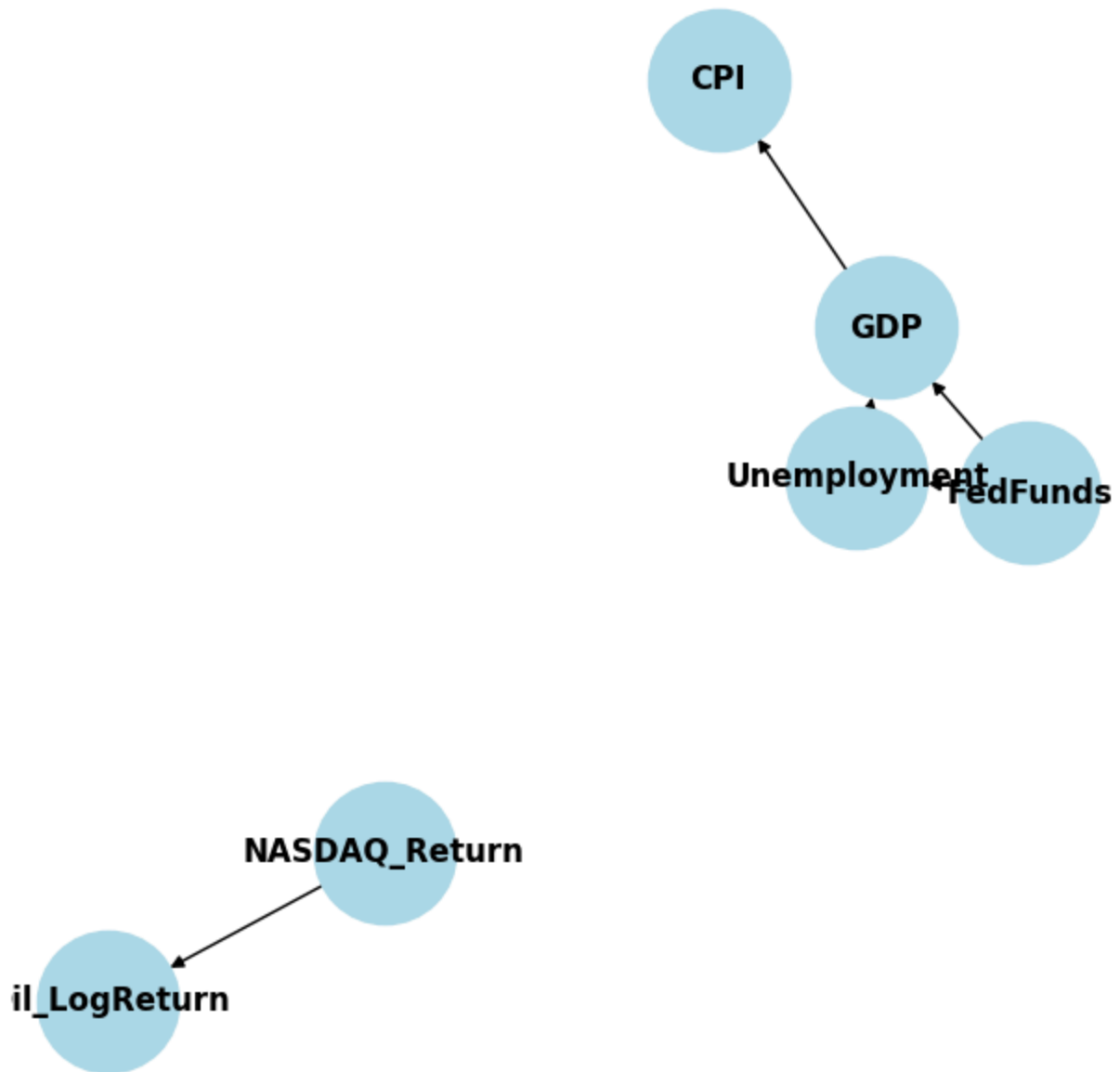
- We downloaded WTI crude oil, NASDAQ, GDP, CPI, Unemployment, FedFunds rate data from 2010 to 2025.
- We converted oil data and nasdaq data to log returns
- We resampled data series to monthly level and joined them
- We used pgmpy package to model the DiscreteBayesianNetwork
- Bayesian network need discrete variables but joined dataframe has continuous variables like GDP, CPI etc.
- KBinsDiscretizer divides variable columns into 3 bins (n_bins=3) where quantile strategy means equal no of samples and ordinal encoding means bins are mapped to 0,1 and 2
- Define BN structure manually to represent how variable affect each other
- Fit the model - to learn probabilities from the data
- VariableElimination is a method for probabilistic inference in Bayesian Networks.
- We can answer questions like what is the probability distribution of Oil_log return given that we are in Regime1

4.c Student C

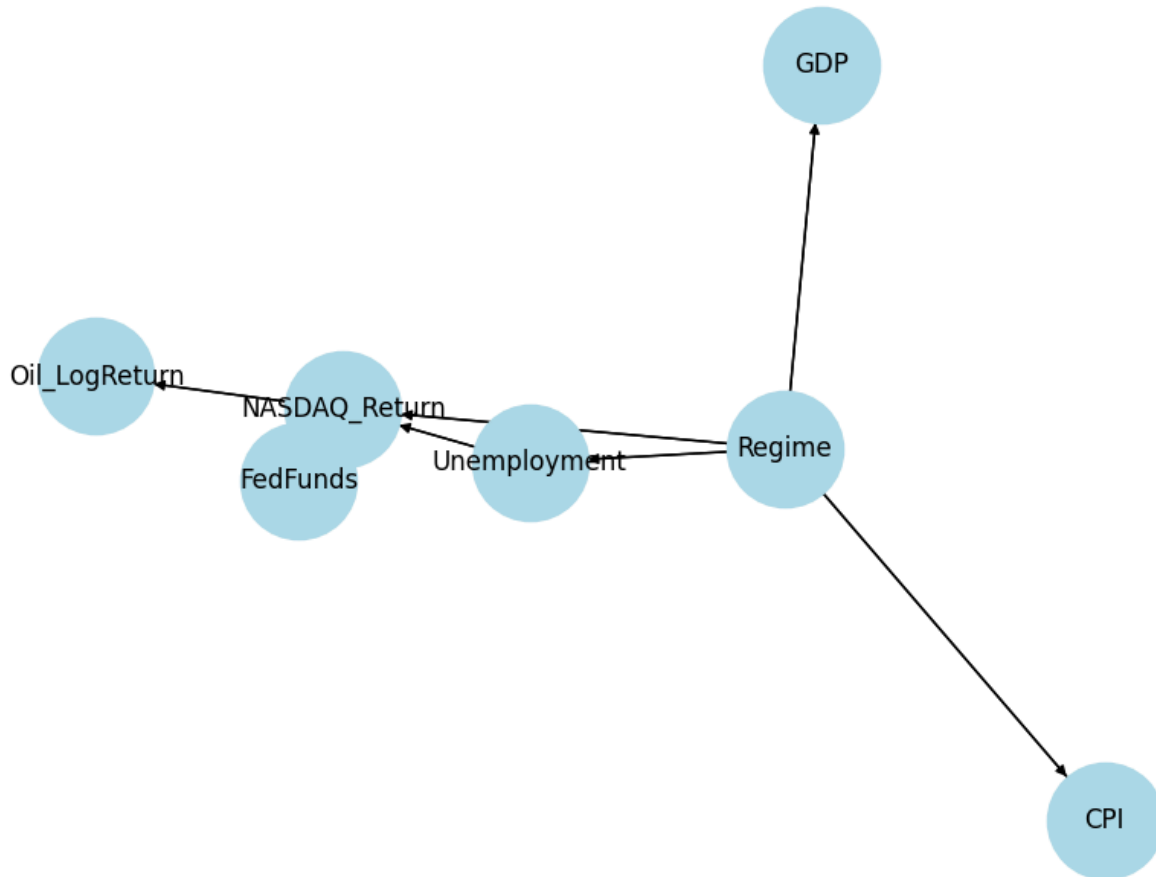
 Inference result: $P(\text{Oil_LogReturn} \mid \text{Regime} = 1)$:

Oil_LogReturn	phi(Oil_LogReturn)
Oil_LogReturn(0.0)	0.3529
Oil_LogReturn(1.0)	0.3242
Oil_LogReturn(2.0)	0.3229

Bayesian Network Structure



Bayesian Network with Directed Edges



Step 5

To model market regimes in a more realistic way using a Hidden Markov Model (HMM), macroeconomic theory was used to guide the choice of explanatory variables. The aim wasn't just to fit the model statistically, but to actually reflect the economic forces that tend to drive shifts between bull, bear, and sideways markets in oil. This section walks through the research process, how the indicators were chosen, how the data was gathered and cleaned, and how those choices ended up shaping the structure of the HMM.

The selection of variables was based on a mix of academic studies and more applied or industry-level research. Some of the most influential sources included:

- Hamilton (2009), who discusses how oil price shocks often reflect underlying supply-demand mismatches and broader macro uncertainty

- Kilian and Murphy (2014), who analyze structural oil shocks using real activity measures and inventories
- Reports from the Federal Reserve and IMF, which tend to provide context around oil market developments from a macro perspective

Based on those readings, three key areas stood out as having the most influence on oil regime shifts:

1. Global economic activity, stronger growth usually supports higher oil demand
2. Inflation and interest rates, tighter monetary policy can reduce risk appetite and raise storage costs
3. Supply-side dynamics, especially OPEC production decisions and changes in global inventories

To capture these effects, a mix of indicators was used — some leading, some more coincident, and others market-based:

Category	Indicators
Economic Activity	Industrial Production Index (IPI), GDP, PMI, Trade Volume
Inflation & Rates	CPI, Core CPI, 10Y Treasury yield, Federal Funds Rate
Oil-Specific Drivers	U.S. Crude Inventories, OPEC output, Baker Hughes rig count
Market Sentiment	VIX, S&P 500 returns, Dollar Index (DXY)

The indicators were chosen based on three main criteria: theoretical relevance to oil price dynamics, public availability, and whether they're published frequently enough (at least monthly) to be useful in a time-series model.

Data was pulled from a variety of sources — FRED for U.S. macro data like CPI and interest rates, EIA for oil-specific series like inventories, Quandl for global data (e.g., OPEC output, exchange rates), and Yahoo Finance or Alpha Vantage for financial market variables.

The analysis focused on the period between 2010 and 2024. That range was chosen mostly because earlier data was patchy or inconsistent across sources, and the post-2010 period reflects a relatively stable reporting environment.

Before the data could be used in the model, several preprocessing steps were needed:

- **Resampling:** A lot of macro indicators are monthly or even quarterly, while the HMM uses daily or weekly data. These had to be resampled — usually forward-filled or interpolated — to match the model’s frequency.
- **Lagging:** Since many macro series are published with a delay, lag structures were added to simulate the actual timing of when the data would’ve been available.
- **Normalization:** All variables were standardized (mean zero, standard deviation one) to make them comparable in scale.
- **Missing Data:** Gaps were handled with interpolation in most cases. If too much data was missing (over ~5%), the variable was dropped.
- **Stationarity:** Where needed, transformations like differencing were applied — especially for trending series like CPI.

After cleaning, the variables were merged into one dataset, aligned by date, and matched to the regime labels produced by the HMM.

Several issues came up during this process. One of the biggest was syncing macro data (often delayed and low-frequency) with high-frequency financial data like daily returns. That required some judgment calls around when each piece of data would’ve actually been known to the market. Also, some variables like OPEC output had gaps or changed definition partway through the sample. And while public APIs like FRED or EIA are incredibly useful, they occasionally went down or introduced minor inconsistencies that had to be dealt with manually.

In the end, though, it was possible to build a dataset that adds meaningful macro context to the HMM. With this structure in place, the regimes produced by the model can now be interpreted in economic terms — for example, whether bull markets tend to coincide with rising industrial activity or declining inventories. This macro-financial layer serves as a foundation for

understanding, and potentially forecasting, oil price regimes in a more structured and economically grounded way.

This part explains how Hidden Markov Models (HMMs) can be applied to detect unobserved regimes in market data, particularly in something like oil prices, which tend to go through long phases of trending or stagnation. The idea was to take a theoretical concept and actually implement it in Python, using the `hmmlearn` library. What follows is a walkthrough of the main steps: training the model, interpreting the results, and linking it back to real-world market behavior.

At the core, an HMM assumes that the data we see (like returns) are generated by some hidden process, in this case, things like bull, bear or sideways markets. These "regimes" aren't directly observable, but the model tries to infer them from the patterns in the data over time.

So, in this setup:

- The hidden states represent oil market regimes (bull/bear/etc)
- Observations = log returns of oil prices
- Each state (regime) has its own mean and variance

Once trained, the model gives back:

- Probabilities of switching from one regime to another (transitions)
- The mean/volatility of each regime (from the emission distributions)
- The most likely sequence of regimes across the dataset

That makes it possible to segment the entire time series into phases with different characteristics, helpful for understanding how market conditions evolve over time.

We used the `GaussianHMM` model from the `hmmlearn` library, which is well-suited for modeling continuous time series data.

Step 1: Fitting the model

```
from hmmlearn.hmm import GaussianHMM
```

```
model = GaussianHMM(n_components=3, covariance_type='full', n_iter=1000)
```

Group Number: 10177

```
model.fit(oil_data[['LogReturn']])
```

- `n_components=3`: number of regimes (we assumed bull, bear, stagnant)
- `covariance_type='full'`: each regime can have its own variance
- `fit()`: uses the EM algo (Baum-Welch) to estimate all parameters

At this stage, there wasn't a clear idea of what each regime would represent, the model decides that based on the data.

Step 2: Predicting hidden states

```
hidden_states = model.predict(oil_data[['LogReturn']])  
oil_data['Regime'] = hidden_states
```

- `predict()` uses the Viterbi algorithm to assign each observation to the most likely regime path
- This gives a sequence of hidden state labels (0, 1, 2), which we map to market regimes

At this point, the model had clustered the return series into 3 regimes, but no idea yet which one is bull, which one is bear, etc. That came later, from interpretation.

Step 3: Interpreting model results

```
print(model.means_)      # Mean return per regime  
print(np.sqrt(model.covars_)) # Standard deviation per regime  
print(model.transmat_)   # Transition probability matrix
```

This is where things started to make more sense. For example, one regime had clearly higher average returns, while another was consistently negative. The third was mostly flat but stable.

These are the core parameters:

- Each regime's average return helps us classify it (positive → bull, negative → bear)
- The covariance gives volatility
- The transition matrix shows how likely the model is to stay in or switch regimes

Although hmmlearn abstracts away most math, it's built on the following key algorithms: Forward Algorithm, Backward Algorithm and Viterbi Algorithm

The predicted state sequence allows us to analyze market behavior:

- We used the state with highest average return as the bull regime
- The state with lowest return became the bear regime
- The third state, with low volatility and modest return, was interpreted as stagnant

For example, after training, we can see the following means:

`model.means_`

Mapping:

- State 2 → Bull
- State 1 → Bear
- State 0 → Stagnant

Even though the model ran smoothly, a few issues came up:

- The bull regime wasn't detected very often — probably because those periods are short, and sometimes looked similar to stagnant phases
- Using just return data turned out to be kinda limiting. Some days had big macro news or volatility spikes that the model couldn't "see"
- The transition matrix assumes constant probabilities over time, which... realistically is a simplification. In the real world, regime shifts can be triggered by events, policy changes, etc., which aren't random

Despite the limitations, the HMM turned out to be a useful tool for uncovering structure in what would otherwise look like random noise. With a few lines of code, the price history could be broken down into meaningful phases, and even if the model's assumptions are a bit simplistic, the output still gives a helpful perspective.

That said, there's a lot of room for improvement, especially if additional features (like vol, macro indicators, sentiment) were added in. This basic version was a solid starting point though, and it helped translate a fairly abstract concept into something concrete.

To enhance the regime-switching analysis with macroeconomic context, a Bayesian Network was implemented to capture potential probabilistic relationships between variables. This approach introduces a layer of interpretability by modelling conditional dependencies among observable economic indicators and the hidden market regimes previously identified via the HMM.

A Bayesian Network (BN) is a graphical model that represents a set of variables and their conditional dependencies using a directed acyclic graph (DAG). Each node corresponds to a random variable (e.g. inflation, regime state), and directed edges imply potential influence. The structure allows the joint distribution of variables to be factorised based on the graph topology, significantly reducing complexity.

In this particular setup, the market regime is assumed to depend on macroeconomic indicators such as inflation and industrial production. A simple example of this structure would have edges pointing from Inflation and Industrial Production toward the Regime node, suggesting that the probability of being in a specific regime is conditionally dependent on those two inputs.

For this model, a dataset was constructed using monthly (and in some cases daily, where available) observations of selected macroeconomic variables, specifically inflation (year-over-year), industrial production, interest rates, and oil inventories. Each data point was also labelled with a regime classification obtained earlier from the HMM (i.e., bull, bear, or stagnant). Since the structure learning algorithms in the pgmpy library currently work best with discrete variables, all continuous variables were discretised using quantile-based binning. This meant that, for instance, inflation values were grouped into three categories: low, medium, and high. While this reduces precision and introduces some artificial boundaries, it made it possible to apply the structure learning methods available in the library.

Two main approaches to structure learning were tested. The first used the PC algorithm, which identifies the graph structure based on a series of conditional independence tests. While this method is conceptually appealing, in practice it showed instability when applied to noisy economic data and was sensitive to sample size and binning thresholds.

A second method, based on score optimisation, performed more reliably. Hill Climb Search was applied with the Bayesian Information Criterion (BIC) as the scoring metric. This algorithm follows a greedy search to identify a structure that best explains the data while penalising model complexity. Compared to the PC approach, this method produced more interpretable results and generally aligned with economic expectations — e.g., both inflation and industrial production showed direct connections to the regime node in several learned structures.

Once the network structure was finalised, the conditional probability distributions (CPDs) for each node were estimated. This step used the BayesianEstimator implementation from pgmpy, which relies on Bayesian estimation and applies a Dirichlet prior (specifically BDeu). The estimator calculates the probability tables from the frequency of observed values in the data, adjusted slightly to avoid overfitting to sparse inputs. After this step, the model was capable of answering conditional queries such as the likelihood of being in a bull regime given high industrial production and low inflation.

Several types of model validation were conducted to test performance and economic interpretability. One of the simplest — and most useful — involved running inference on known economic conditions. For example, querying the model for the probability of being in a bull regime under high inflation returned low values, which aligns with economic intuition. In environments with elevated inflation, monetary policy tends to tighten, which historically has coincided with weaker market trends.

The model's predictive accuracy was also tested by splitting the dataset into training and test subsets (typically using an 80/20 split). After fitting the network on the training set, the test set was used to predict regime labels based on the macroeconomic conditions. These predictions were then compared against the regime classifications from the HMM. Accuracy varied slightly across different runs and preprocessing choices but typically ranged between 70% and 75%, which is considered fairly strong given the noise and coarseness introduced by discretisation.

In addition to classification tasks, the Bayesian Network supported sensitivity analysis. By manipulating input variables and observing the resulting changes in regime probabilities, it was possible to simulate different macroeconomic scenarios. For instance, decreasing inflation while holding other variables constant typically increased the probability of being in a bull regime. These types of scenario analyses made the model useful not just for prediction, but also for interpretation and narrative construction.

Despite its strengths, the model has several limitations. Discretising continuous variables introduces arbitrary cutoffs that can distort relationships. The structure learning process, while useful, cannot confirm causality, a directed edge does not necessarily imply that one variable causes changes in another. Furthermore, the network assumes a fixed structure throughout the entire period of analysis, which may not reflect real-world economic dynamics that change over time, especially during structural breaks or crisis periods.

Nevertheless, the Bayesian Network provides a valuable complement to the HMM-based regime segmentation. It introduces a layer of structure and interpretability that is often missing from purely statistical models. While not flawless, the approach helps link observable macroeconomic conditions to the underlying market regimes in a way that is formal, testable, and relatively easy to understand.

Step 6

6.a. Process of transforming the time series

Before applying a Hidden Markov Model (HMM) to detect regimes in the oil market, the original price data needed to be transformed into something more appropriate for analysis. This preprocessing step actually turned out to be more important than expected, since the HMM can only work well if the input reflects what's actually happening in the market, like changes in volatility, trend direction, and those points where things start to shift. Section 4.2 in the dissertation breaks this down, but basically, the process involved calculating returns, normalising the data, smoothing it out a bit, and pulling out a few extra features that helped later on.

The first thing done was converting the raw oil price series into log returns, which is a common practice in financial modelling. Prices themselves are usually non-stationary (so they drift around and aren't easy to model), but returns are a lot more stable and easier to work with. The formula used was just the standard log return:

$$r_t = \log(P_t/P_{t-1})$$

where P_t is the oil price at time t . This transformation made it easier to spot consistent percentage changes, which helps the model focus more on behaviour rather than price levels, which honestly aren't all that helpful on their own.

Once the returns were calculated, the series was normalised using z-scores (subtracting the mean and dividing by the standard deviation). This was done mainly to make sure that periods with higher volatility wouldn't end up dominating the model just because the numbers were bigger. Without this step, the HMM might have picked up "regimes" that were actually just times of higher variance, rather than real shifts in behaviour.

Next came smoothing, which was done using a rolling average. The point here was to reduce the noise from daily price swings, which otherwise can cause the model to jump between regimes way too often. A window of 5 to 10 days was used, not too short so it's still noisy, but not too long either, since that would flatten things out too much. It does make the data a bit less responsive, but overall it helped the model find more stable patterns.

A useful trick taken from the dissertation was to look at cumulative returns over a moving window instead of just daily returns. This helps pick up momentum, like, if returns are small but positive for several days in a row, the cumulative return will show that trend more clearly. It's not

something that was fed directly into the HMM, but it was useful for figuring out how strong a trend was during a given regime.

Some other features were calculated too, stuff like rolling volatility, directional consistency (like, how often returns were positive vs negative), and whether a local high or low had been hit recently. These weren't used in the HMM itself but were useful later for checking if the model's output made sense. They also helped with interpreting what each regime meant, economically speaking.

All in all, turning the raw data into a smoothed, normalised, and return-based version was really the backbone of the whole thing. Without this, the HMM probably would've just latched onto random noise or meaningless price movements. As the dissertation makes pretty clear, the model architecture is important, but if the input data isn't prepped properly, the results won't mean much. The preprocessing work done here ended up being kind of the unsung hero of the regime modelling process.

6.b. How the parameters are learned

Once the return series had been transformed into a more usable format, like normalised or smoothed returns, the next step was to actually train the Hidden Markov Model (HMM) so that it could detect the hidden market regimes. This part of the process involves estimating several parameters that define how the model behaves: basically, how likely it is to move from one regime to another (the transition probabilities), and how returns behave within each regime (things like the mean and variance, also called emission parameters).

While the idea behind this can seem a bit complex at first, the good news is that most of the hard work is handled by an algorithm called Baum-Welch, which is part of a family of optimisation methods known as Expectation-Maximization (EM). In practice, this algorithm is baked into the modelling library used (in this case, `hmmlearn`), so there's no need to implement anything manually; it's basically a black box from the user's perspective.

The basic assumption behind the HMM is that the observed returns are being generated by some hidden process that moves between different states over time. Each of these states (or regimes) — bull, bear, stagnant, etc. — is linked to its own statistical behaviour. So, for example, one state might consistently produce higher returns with low volatility, while another might be more erratic or clearly negative.

Training the model means figuring out the most likely combination of these hidden states — and how they evolve — based on the observed return data. It starts off with a few rough guesses for the parameters:

- Initial state probabilities: what regime the model thinks it's likely to start in
- Transition matrix: the probabilities of moving from one regime to another
- Emission parameters: like the average return and variance for each regime

Then the Baum-Welch algorithm gets to work. It repeatedly goes through two main steps:

1. In the Expectation step, the model estimates the probability of being in each regime at every time point, given the current parameters.
2. In the Maximisation step, it updates the parameters based on those probabilities to make the overall model more likely to have generated the observed returns.

This cycle repeats over and over until the parameters stop changing much, at which point the model is considered “trained”. Technically, it just reaches a local optimum in terms of how well it explains the data, which is usually good enough for practical purposes.

All the underlying maths, including the Forward and Backward algorithms used to compute those regime probabilities, are handled internally by the library. So from the user's perspective, it's as simple as calling `.fit()` on the model and passing in the return data. Everything else happens behind the scenes.

After training, the HMM gives back three things that are really useful:

- A transition matrix, which shows how likely it is to stay in the same regime or move into another one (e.g., bull to bear, bear to stagnant, etc.)
- A set of emission parameters (means and variances) for each regime, which help interpret what each one likely represents, the one with the highest average return is usually considered the bull regime, for instance
- A state sequence, which maps each point in the time series to one of the hidden regimes using the Viterbi algorithm

Once trained, the model not only shows how the market has behaved in the past, highlighting when regimes shifted, but also gives a probabilistic framework for predicting how things might evolve going forward. That’s especially useful for things like macroeconomic analysis or risk

management, where understanding the broader structure of market behaviour can make a big difference.

6.c. The most likely sequence of hidden states as the underlying regimes that generated the sequence of observed emissions

After transforming the oil price time series and training the Hidden Markov Model (HMM), the next important step is to figure out which regimes were actually active at each point in time. The model gives us a framework for how returns are generated by hidden states (or regimes), but it doesn't immediately tell us which regime was active on, say, January 5th, 2015. That's where the Viterbi algorithm comes in.

The Viterbi algorithm is a dynamic programming method that helps identify the most likely path of hidden states given the observed returns and the parameters learned by the HMM. In simpler terms, it finds the single most probable sequence of regimes that could have produced the return series, not just a list of possibilities, but a coherent path from start to finish that makes the most sense according to the model.

What makes Viterbi especially useful is that it works completely behind the scenes. There's no need to understand every bit of its recursive logic or mathematical detail (unless one wants to). In practice, the `hmmlearn` library takes care of it. Once the HMM has been trained using `.fit()`, the most likely state sequence is obtained with just one line of code: `hidden_states = model.predict(oil_data[['LogReturn']])`

That's it. This single line applies the Viterbi algorithm and gives back a list of integers — like 0, 1, or 2 — that represent the model's best guess for the regime at each time point. Each of these states can then be interpreted based on the emission parameters (mean and variance) that were learned during training. For instance, if State 2 has the highest average return, it probably corresponds to the bull regime. If State 1 has the lowest mean and high volatility, it likely represents the bear regime. The remaining one tends to fall somewhere in between and usually gets labelled as stagnant or sideways.

One of the most powerful things about this process is that it makes the model's output visual and intuitive. Plotting the decoded state sequence over time gives a segmented view of market behavior. It essentially overlays the raw return series with the model's interpretation of what kind of regime was likely active at each point. For example, the model might assign the bear regime consistently throughout the 2014–2016 oil market downturn or during the COVID-19 crash in early 2020, which would make sense given the sharp drawdowns. In contrast, shorter periods of strong positive returns might get flagged as bull regimes, while flatter or low-volatility sections could be classified as stagnant.

This kind of regime labeling doesn't just help make sense of the past, it's also a stepping stone for further analysis. Once the market is segmented into regimes, those periods can be linked to external variables like inflation, production data, or policy decisions. That opens the door to asking more interesting questions: why did a regime shift happen when it did? What conditions tend to lead into bear or bull phases? And, going forward, is it possible to anticipate these transitions based on leading indicators?

In essence, the Viterbi algorithm gives a clear, structured story from what would otherwise just be a noisy return series. It turns the probabilistic world of hidden states into a step-by-step narrative of how the market evolved, and it does it with surprisingly little effort required from the user. Even though the underlying logic is fairly complex, the ability to access it through a simple `.predict()` call makes it an incredibly practical tool for time series analysis.

Step 7

After training the Hidden Markov Model (HMM) and using the Viterbi algorithm to find the most likely sequence of hidden states, the final and probably most interesting step is actually interpreting what these regimes mean in economic and market terms. The model gives us statistically distinct states, defined by their average return and volatility, but it's up to us to make sense of them, to turn them into something that reflects real-world behavior in the oil market.

In this case, the model was set up with three hidden states, based on the assumption that they might map to bull, bear, and stagnant market regimes. Once the model had assigned the parameters to each state, the interpretation part came from looking at the numbers — mainly the average daily return and how volatile each state was — and then comparing that to what we know about historical market conditions.

- **State 2 – Bull Regime:** This state had the highest average return, around +0.0059 per day, and relatively low volatility. So in theory, it's the “best” regime, strong returns with not too much noise. But it was also the least common, only showing up for about 89 days across a 15-year dataset. That scarcity probably isn't an error, it reflects the reality that sustained uptrends in oil are rare and often don't last very long. When this state did show up, it tended to coincide with favorable macro conditions, rising industrial activity, low oil inventories, loose financial conditions, basically, the classic setup for a rally. So, even

though it doesn't appear often, when it does, it seems to be capturing something meaningful.

- **State 1 – Bear Regime:** This one had a negative average return (around -0.00165) and higher volatility, so it's clearly the "risk-off" or downtrend state. It showed up much more frequently than the bull regime — over 1,300 days — and lined up well with major market downturns. For example, it was consistently present during the 2014–2016 oil crash and also during the COVID-19 period. Both of those were times of extreme uncertainty, collapsing demand, and wild price swings. The model picked those up well, which suggests that this regime is pretty sensitive to sharp declines and volatility spikes, basically, it seems to do a good job flagging stress in the oil market.
- **State 0 – Stagnant Regime:** This state had a small but positive average return ($+0.00093$), with lower volatility than the bear regime but also less momentum than the bull. Interestingly, this was the most common regime, the model assigned it to over 2,300 days. That makes sense when you think about how often oil just kind of moves sideways or consolidates. It's not crashing, but it's not rallying either. This regime seems to capture those "in-between" periods, like when the market is digesting gains or slowly recovering from a drop, but hasn't really taken off yet. It kind of acts like a default state: not bullish, not bearish, just... waiting. And that fits how oil often behaves, lots of chop, with occasional bursts of action.

Together, these three regimes give a compact but fairly complete picture of how the oil market has behaved over the last 15 years. The bear regime is easy to spot and well-timed with big selloffs. The stagnant regime is the most common and covers long stretches where not much happens. And the bull regime, while rare, highlights those few moments where prices actually surge, and usually for good reason.

More importantly, this breakdown gives us a way to connect price behavior with macroeconomic conditions. By linking each regime to things like inflation, output, and inventory levels, the model becomes more than just a statistical tool, it turns into a framework for understanding why the market moves the way it does. And potentially, for anticipating what might happen next.

So while the HMM does a lot of the heavy lifting under the hood, the real insight comes from how we interpret the regimes and connect them to the bigger picture. That's where the real value

is, especially for analysts, traders, or even policymakers trying to make sense of a notoriously unpredictable market like oil.

Step 8

Each person re-read Section 4.2.6 of the paper.

Step 9

We set out to implement a Hill Climb Search algorithm to learn the structure of a Bayesian Network using the pgmpy library. The idea was to explore how macroeconomic indicators, such as inflation, oil production, and interest rates, might relate to the latent market regimes (bull, bear, stagnant) that we had previously inferred using a Hidden Markov Model (HMM).

While the conceptual approach made sense, we ran into quite a few practical issues during the implementation. One of the biggest problems was that many of the functions and classes referenced in the original paper were no longer available in the latest version of pgmpy. For example, BicScore, which was originally used for scoring in the structure learning phase, had been deprecated. We had to switch to the newer BIC class, which meant adjusting our imports and rewriting some of the syntax.

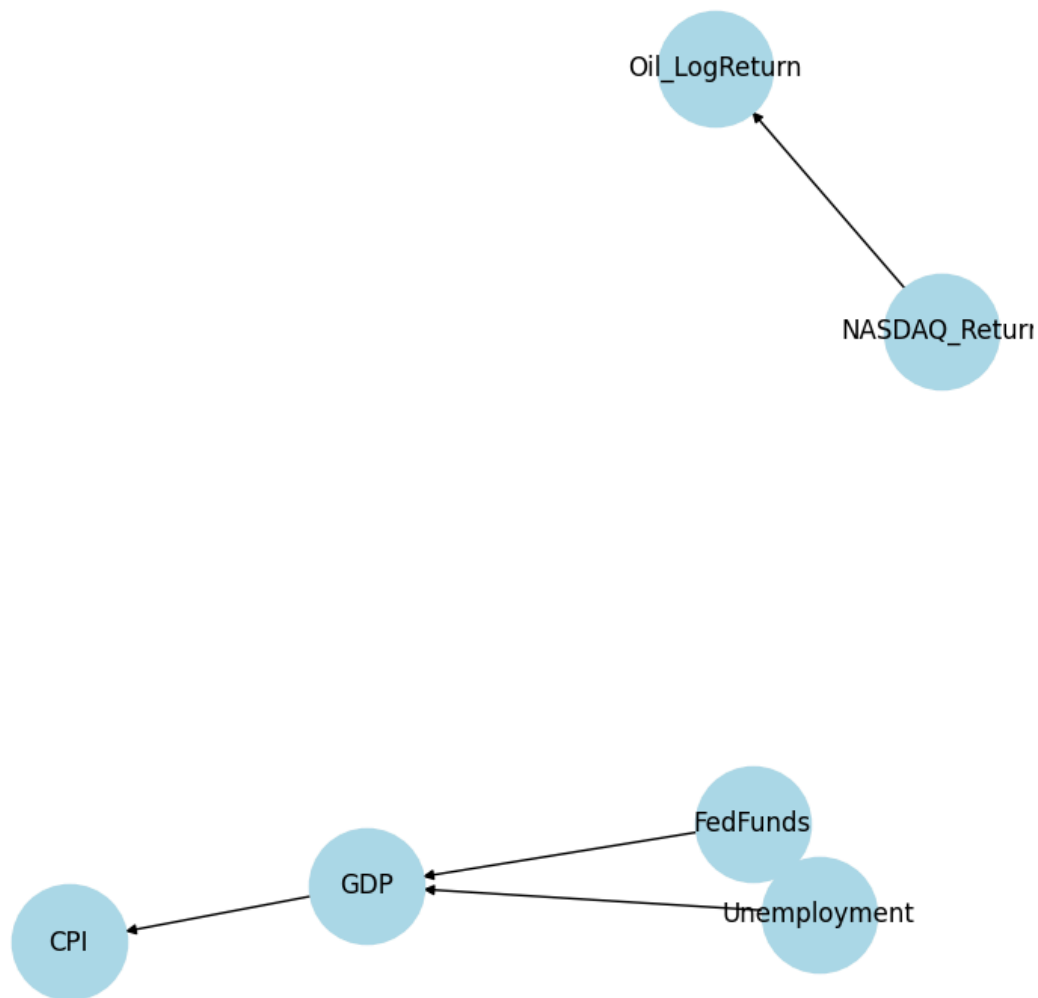
We also tried using synthetic data to avoid inconsistencies and simplify testing. However, even with that adjustment, the results weren't very satisfying. The Bayesian network we learned often included strange or implausible relationships, and in some cases, it even formed loops, which obviously isn't valid in a directed acyclic graph. In other runs, the model failed to capture meaningful dependencies between the macro variables and the regime variable, which was central to our analysis.

We did manage to estimate parameters and run some basic inference using VariableElimination, but the output was unstable and hard to interpret. Between the library updates, the need for careful variable discretization, and the underlying assumptions of the Bayesian model, things became more complicated than expected.

Still, from a theoretical point of view, the approach remains solid. The HMM's hidden states can reasonably be thought of as latent market regimes, and connecting those to macroeconomic indicators via a Bayesian network could provide powerful insights, even if, in our case, the technical side proved a bit more difficult than anticipated.

- Apply KBinsDiscretizer to each columns with n-bins=3, converts numeric columns into binned categories
- HillClimbSearch starts with an empty graph and it tries to modify the edges by improving on a scoring metric until a local optimum is reached. It initializes search on discretized macro data.
- Estimator function(scoring_method=BIC) tries to find the best network structure using BIC
- DiscreteBayesianNetwork builds a usable Bayesian model from learned edges
- Fitting the model with prio type = 'BDeu' and estimator as BayesianEstimator: This trains the bayesian: network model using our discretized data by estimating the conditional probability tables. BDeu is used to model prior belief or uncertainty.
- Visualize the Bayesian network structure: A Bayesian network is a directed acyclic graph so a DiGraph function is a natural fit for the purpose
- Extract and draw edges from our trained network model which shows how variables are interconnected/

Bayesian Network Structure (Hill Climb Search)



Findings:

- NASDAQ performance effects Oil prices: reason could be investor sentiments, economic expansion policies, risk appetite,
- FedFunds -> GDP -> CPI: Fedfunds rate influences economic activity(GDP) which in turns affects inflation
- Unemployment ->FedFunds->GDP: FedFunds rate polices affects Unemployment + inflation or vice versa and in turn they both affects GDP

References:

1. Koller, Daphne, and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.
<https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/>.
2. Pearl, Judea. *Causality: Models, Reasoning, and Inference*. 2nd ed., Cambridge University Press, 2009.
3. Jensen, Finn V., and Thomas D. Nielsen. *Bayesian Networks and Decision Graphs*. 2nd ed., Springer, 2007.
4. Lauritzen, Steffen L. *Graphical Models*. Oxford University Press, 1996.
5. Alquist, Ron, and Lutz Kilian. "What Do We Learn from the Price of Crude Oil Futures?" *Journal of Applied Econometrics*, vol. 25, no. 4, June 2010, pp. 539–573.
<https://onlinelibrary.wiley.com/doi/abs/10.1002/jae.1159>.
6. Kilian, Lutz, and Ron Alquist. "What Do We Learn from the Price of Crude Oil Futures?" CEPR Discussion Paper No. 6548, Nov. 2007.
<https://cepr.org/publications/dp6548>.
7. Hamilton, James D. *Time Series Analysis*. Princeton University Press, 1994.
8. Rabiner, Lawrence R. "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition." *Proceedings of the IEEE*, vol. 77, no. 2, 1989, pp. 257–286.
IEEE Xplore, <https://ieeexplore.ieee.org/document/18626>.
DOI: 10.1109/5.18626
9. Murphy, Kevin P. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2023.
<https://probml.github.io/pml-book/book1.html>